

Database Architecture & ERD Design Report (Phase 1)

Sports Event Ticket Booking & Reservation Platform

Technical Documentation | Relational Database Systems

Database Engine: PostgreSQL 16+ (UTF-8)	Normalization Target: Third Normal Form (3NF)
Platform Type: High-Concurrency Ticketing System	Architecture Pattern: Vertical Table Extension

1. Introduction & Architectural Objectives

The primary objective of Phase 1 is to perform a domain analysis for a high-traffic sports ticketing platform and construct an optimal, fully normalized relational schema. The architecture eliminates data redundancies and update anomalies by strictly adhering to Third Normal Form (3NF). It establishes a robust foundation for high-throughput transactional processing (ACID) and efficient query execution under heavy concurrent traffic.

2. Database Entities & System Schema Overview

The system architecture comprises **8 interconnected, domain-partitioned tables**:

- users Table:** Stores user demographics, hashed credentials (`password_hash`), roles (`role`), and account status flags. To satisfy 1NF atomicity, names are decomposed into `first_name` and `last_name`.
- tickets Table:** Generalized base entity storing shared match attributes including `home_team`, `away_team`, `sport_type`, `ticket_tier`, `organizer`, `venue_name`, `city`, `match_date`, `price`, and `remaining_capacity`.
- Sport Extension Tables** (`football_details`, `volleyball_details`, `basketball_details`): Specialized 1 : 1 child entities isolating discipline-specific metadata without polluting the base `tickets` table with excessive NULL values.
- reservations Table:** Controls temporary ticket holds with expiration timeouts (`expires_at`) prior to payment.
- payments Table:** Captures transaction ledgers linked via a 1 : 1 relation with reservations for financial integrity.
- reports Table:** Logs customer support issues using dual nullable foreign keys (`ticket_id` and `reservation_id`).

3. Normalization Analysis & 3NF Justification

All 8 schema entities satisfy Third Normal Form (3NF) standards:

- First Normal Form (1NF):** Attributes are strictly atomic (`first_name`, `last_name`, `home_team`, `away_team`). Multi-valued attributes and repeating groups are converted into normalized child extension tables.
- Second Normal Form (2NF):** Fulfills 1NF. Non-prime attributes are fully functionally dependent on primary keys (*PK*). Extension tables rely on surrogate primary key `ticket_id`, eliminating partial dependencies.
- Third Normal Form (3NF):** Contains zero transitive functional dependencies ($X \rightarrow Y \rightarrow Z$). Specialized sport metadata is vertically partitioned into dedicated 1 : 1 extension tables, eliminating sparse, NULL-heavy base tables.

4. Schema Architecture Summary

The following table provides a comprehensive architectural summary of all system entities, primary key identifiers, foreign key dependencies, and their underlying 3NF normalization rationales:

Table Name	Primary Key	Foreign Key(s)	3NF Design Rationale & Architectural Purpose
users	user_id	None	Enforces unique email/phone constraints; atomic name fields; decouples authentication credentials.
tickets	ticket_id	None	Generalized match metadata shared across all sports; decouples team names into home_team and away_team.
football_details	ticket_id	ticket_id → tickets	1 : 1 extension isolating stadium name, stand section, row/seat positioning, ticket type, and outdoor amenities.
volleyball_details	ticket_id	ticket_id → tickets	1 : 1 extension isolating indoor hall layout, seating section, row/seat allocation, ticket tier, and hall amenities.
basketball_details	ticket_id	ticket_id → tickets	1 : 1 extension isolating arena layout, seating section, row/seat allocation, ticket tier, and court-side amenities.
reservations	reservation_id	user_id, ticket_id	Manages transient ticket hold lifecycles and expiration logic without redundant match or spectator data storage.
payments	payment_id	reservation_id, user_id	1 : 1 financial transaction ledger fully decoupled from transient reservation state to ensure auditability.
reports	report_id	user_id, ticket_id, reservation_id	Independent customer support logging; uses dual nullable foreign keys to reference either tickets or reservations.

5. Integrity Constraints & Domain Business Logic

To enforce data consistency at the database engine layer and prevent invalid state transitions, declarative constraints and PostgreSQL custom domain enumerations are defined:

- **Numeric & Value Invariants (CHECK):**
 - Non-negative ticket pricing: Enforced via CHECK (`price >= 0`) to eliminate negative billing anomalies.
 - Positive venue capacity: Maintained via CHECK (`remaining_capacity >= 0`) to guarantee seat allocation boundaries.
 - Non-negative settlement amounts: Enforced via CHECK (`amount >= 0`) in the financial transaction ledger.
- **Temporal Consistency & Expiration Logic:**
 - Valid reservation timeframe validation: Enforced via CHECK (`reserved_at < expires_at`), guaranteeing that expiration timeouts are strictly chronological and enabling automated cleanup triggers.
- **Domain Enumerations (ENUM):**
 - User Roles (`user_role`): Bounded domain restricted to 'audience' and 'support'.
 - Reservation Lifecycle Status (`reservation_status`): Domain constrained to 'pending', 'paid', and 'cancelled'.
 - Financial Payment Status (`payment_status`): Domain restricted to 'successful', 'failed', and 'pending'.
 - Sport Category Enumeration (`sport_type_enum`): Domain bounded to 'football', 'volleyball', and 'basketball'.
- **Referential Integrity & Cascading Actions:**
 - Account Cleanup (ON DELETE CASCADE): Automatically purges associated reservations when a user profile is deleted.
 - Audit Preservation (ON DELETE SET NULL): Preserves customer support dispute logs when referenced ticket or reservation records are modified or purged.

6. Performance Indexing & Query Optimization Strategy

To accelerate high-frequency queries, multi-attribute wildcard lookups, and concurrent event searches, a multi-tiered indexing strategy is implemented:

- **Compound B-Tree Indexes:**
 - Multi-Column Filter Index: Constructed on (`sport_type`, `city`) to optimize composite filtering during active ticket discovery operations.
 - Chronological Order Index: Configured on `match_date` ASC to facilitate fast $O(\log N)$ time range scans and chronological match listings.
- **GIN Trigram Indexes (pg_trgm):**
 - Generalized Inverted Index (GIN) trigram indexes configured on `venue_name`, `home_team`, and `away_team`.
 - Accelerates sub-string wildcard searches (`ILIKE '%term%'`) across team names and venue titles, reducing lookup time complexity from $O(N)$ full table scans to $O(\log N)$ inverted index scans.
- **Query Execution Plan Optimizations:**
 - Primary Key B-Tree Indexing: Automatically generated B-Tree indexes on all surrogate primary keys (`user_id`, `ticket_id`, `reservation_id`, `payment_id`, `report_id`).
 - Unique Indexes: Enforced on `phone_number`, `email`, and `payments.reservation_id` to guarantee fast $O(1)$ constraint checks during user login and payment verification.

7. Entity-Relationship Data Dictionary & Structural Analysis

The complete structural breakdown of all system entities, primary attributes, foreign key relationships, and cardinalities is formally detailed in Table 2 below:

Entity / Table	Attributes & Keys	Entity Relationships	Cardinality & Target
users	user_id (PK), first_name, last_name, phone_number, email, password_hash, role, city, is_active, created_at	Makes Reservation Processes Payment Submits Support Report	1 : N → reservations 1 : N → payments 1 : N → reports
tickets	ticket_id (PK), home_team, away_team, sport_type, ticket_tier, organizer, venue_name, city, match_date, price, remaining_capacity, is_active, created_at	Booked In Reservation Has Football Details Has Volleyball Details Has Basketball Details	1 : N → reservations 1 : 1 → football_details 1 : 1 → volleyball_details 1 : 1 → basketball_details
football_details	ticket_id (PK/FK), league_name, stadium_name, stand_section, row_number, seat_number, ticket_type, amenities	Extension of Base Ticket Entity	1 : 1 → tickets
volleyball_details	ticket_id (PK/FK), league_name, hall_name, seat_section, row_number, seat_number, ticket_tier, amenities	Extension of Base Ticket Entity	1 : 1 → tickets
basketball_details	ticket_id (PK/FK), league_name, hall_name, seat_section, row_number, seat_number, ticket_tier, amenities	Extension of Base Ticket Entity	1 : 1 → tickets
reservations	reservation_id (PK), user_id (FK), ticket_id (FK), status, reserved_at, expires_at	Settled By Payment Ledger Referenced By Support Report	1 : 1 → payments 1 : N → reports
payments	payment_id (PK), reservation_id (FK/Unique), user_id (FK), amount, status, payment_method, paid_at	Settles User Reservation	1 : 1 → reservations
reports	report_id (PK), user_id (FK), ticket_id (FK/Nullable), reservation_id (FK/Nullable), category, report_text, status, created_at	Submitted By User Account Refers To Target Record	N : 1 → users N : 1 → tickets N : 1 → reservations

8. Phase 1 Executive Summary & Architectural Conclusion

Phase 1 successfully delivers a fully normalized Third Normal Form (3NF) database architecture in PostgreSQL, engineered specifically for high-concurrency sports event ticketing and reservation workflows. By conducting an exhaustive domain analysis, all user demographic attributes, base ticket metadata, discipline-specific details, transient holds, financial transaction ledgers, and support ticket logs have been cleanly partitioned into 8 cohesive tables.

8.1 Key Architectural Milestones Achieved

- **Atomic & Normalized Attribute Decomposition:** User identities are split into `first_name` and `last_name`, while match titles are separated into `home_team` and `away_team`, eliminating 1NF non-atomic string structures and enabling indexed team filtering.
- **Extensible 1 : 1 Vertical Table Extensions:** Discipline-specific features for football, volleyball, and basketball are isolated in child extension tables. This architecture prevents base `tickets` table pollution from excessive `NULL` values while allowing seamless addition of future sport types.
- **Engine-Level Data Integrity:** Strict declarative `CHECK` constraints, custom PostgreSQL `ENUM` domains, temporal validity rules (`reserved_at < expires_at`), and cascading foreign key actions preserve transactional consistency without relying solely on application-level validations.
- **High-Performance Indexing Strategy:** Compound B-Tree indexes and GIN Trigram indexes (`pg_trgm`) guarantee $O(\log N)$ query time complexity for range filtering, chronological sorting, and fuzzy sub-string team searches under high concurrent load.

8.2 Production Readiness & Phase 2 Integration

The designed relational schema establishes a robust foundation for Phase 2 backend implementation. The structure is fully compatible with standard PostgreSQL Data Definition Language (DDL) scripts, stored procedures (PL/pgSQL), database triggers for capacity management, and downstream integration with FastAPI web services and Redis caching layers.